

gbrc: Static recompiler for Game Boy

Team Gloog 

20 September 2026

1 Overview

gbrc is a LLVM-based static recompiler for the Game Boy DMG-01 i.e. using gbrc, one can recompile Game Boy games for any system and build their own frontend using the runtime library and play it as a native executable. As of right now, gbrc only supports games which don't require a memory bank controller (MBC).

2 Installation

To install gbrc, run the following command:

```
$ go install github.com/boxmanllc/gbrc@latest
```

3 Usage

The gbrc CLI takes in two required flags, which are `--rom` and `--frontend`. `--rom` is the path to the ROM file and `--frontend` is the path to the object file of the frontend to link against. It also accepts the following optional flags:

- `--out`, which is the directory where the output and the generated IR are written (by default, this is the directory containing the ROM)
- `--target`, which is the target architecture used for cross-compiling (e.g. `wasm`, in which case `emcc` is used instead of `clang`)
- `--profile`, which is the path to a profile file whose recorded program counters are used to seed additional entry points for the control-flow analysis
- `--save-ir` and `--only-ir`, which respectively emit the generated LLVM IR alongside the compiled binary and emit only the IR without compiling it

For example, to recompile `tetris.gb` using the SDL frontend example, run the following commands:

```
$ git clone https://github.com/boxmanllc/gbrc.git
$ clang -O2 -I ./runtime/include $(pkg-config --cflags sdl2) -c ./runtime/examples/sdl/sdl.c -o ./build/sdl.o
$ gbrc --rom ./tetris.gb --out ./build --frontend ./build/sdl.o
```

Once the binary is compiled, it can be executed using `./build/tetris`. Apart from normally running the executable, it can also be run in “profile” mode, where the program counter (PC) is periodically captured and saved to a file, which can then be used as additional entry points during the recompilation step:

```
$ ./build/tetris --profile ./build/tetris.prof
$ gbrc --rom ./tetris.gb --out ./build --frontend ./build/sdl.o --profile ./build/tetris.prof
```

4 Architecture

gbrc isn't a fully self-contained static recompiler, i.e. the LLVM IR emitted at the end doesn't contain the code related to graphics and joypad input handling. It focuses mainly on statically recompiling the actual instruction stream of the game, i.e. the CPU portion. The instruction stream by itself

wouldn't really make any game playable, as other hardware components such as the PPU, APU, timer, joypad and interrupts are crucial for running a Game Boy game. The CPU communicates with all of these other hardware components using memory-mapped I/O, where it reads from and writes to special memory addresses. The behavior of these other hardware components is implemented using an external C runtime library, which is linked at compile time alongside the recompiled code.

Sometimes the game jumps to an address that gbrcc couldn't predict, such as a return from a function or a jump through a register (ex: JP HL). When that happens, the recompiled code hands control to a small interpreter in the runtime library, which keeps running until it reaches a block that was recompiled or the frame's cycle budget runs out.

The entire pipeline of gbrcc can be described using the following flow chart:

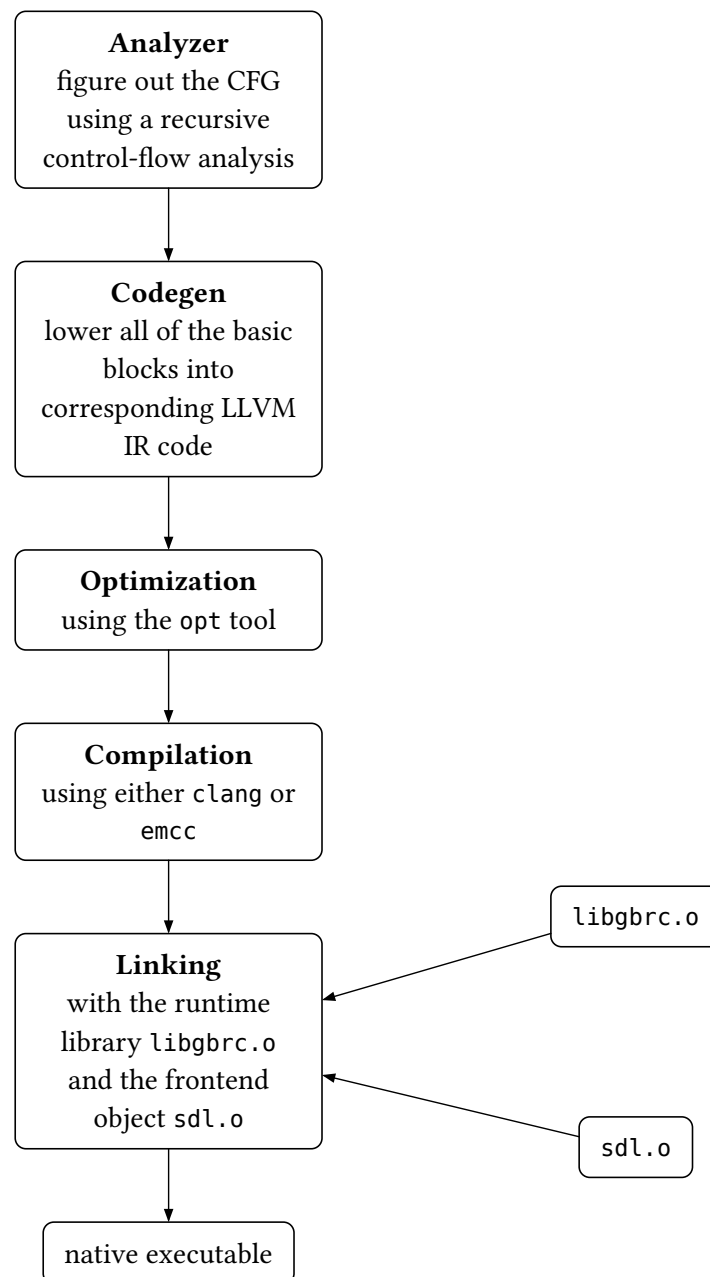


Figure 1: gbrcc's recompilation pipeline

The analyzer module performs a recursive control-flow analysis to recover the structure of the game's code. It starts from known entry points, which include the interrupt handlers, the cartridge's

entry point and user code. It then walks through the code and splits it into basic blocks at every branch, call and return. Anything that isn't encountered by this walk is treated as data rather than code. Apart from the known entry points, additional entry points can be added using the `--profile` flag, as mentioned above in the Usage section.

Each basic block is then lowered into LLVM IR using the [llir/llvm](#) Go library. The entire game loop is represented as a single function, in which each basic block branches to a central dispatcher. Registers, flags, the stack pointer, the program counter and the 64 KiB address space are emitted as globals, and each instruction becomes a call to a small function that operates on them. The dispatcher then jumps to the block matching the program counter, or falls back to the runtime interpreter if it doesn't match any recompiled block.

5 Making your own frontend

A frontend is a small program which implements the hooks declared in `gbrc.h` and is then linked against the runtime library using the `--frontend` flag. The runtime takes care of the CPU and the rest of the hardware, while the frontend takes care of everything platform specific, i.e. creating a window, taking input, presenting the framebuffer and playing audio. To get started, include `gbrc.h` with `GB_IMPLEMENTATION` defined, implement the hooks described below, call `gb_attach` once at startup and then start the game loop using `gb_run`.

The repository ships with an SDL2 desktop frontend and an Emscripten-based browser frontend under [runtime/examples](#), both of which implement the same set of hooks.

The hooks are:

- `gb_init`, which is called once when the game boots. It receives the game's title and is the right place to set up the platform, i.e. creating a window, opening an audio device or registering event listeners. It returns `false` if initialization fails
- `gb_poll`, which is called once per frame to drain the platform's event queue. Input events are translated into calls to `gb_set_button` here, and a quit request is recorded so that `gb_should_quit` can report it
- `gb_should_quit`, which is checked after every `gb_poll` and tells the runtime whether it should stop the game loop
- `gb_prepare_video`, which is called whenever a frame is complete. It receives a 160x144 framebuffer where each byte holds a 2-bit shade index (0 to 3), which the frontend maps to a palette and presents on screen
- `gb_prepare_audio`, which is called with a buffer of interleaved left and right 16-bit samples, along with the number of samples, whenever enough of them have been collected. A frontend can queue them for playback or ignore them entirely
- `gb_wait_frame`, which is called at the end of every frame and is used to pace the loop, i.e. to keep the game from running faster than the real hardware
- `gb_shutdown`, which is called once when the loop ends, and is where any platform resources allocated in `gb_init` should be released